

4 NPU × 1 SSU：从 Path0 stall 到可变请求调度

同输入比较 Baseline、Once-per-layer、deadline CIR 与在线 NPU 分配

2026-09-06 · 原项目仿真器实测

Contents

1 先说结论：找到改善，但没有证实“总是大幅领先”	1
2 这次究竟固定了什么？	2
2.1 变化输入的一个具体参数表	2
2.2 三种“带宽”不要混在一起	2
3 如何计算利用率，避免挑窗口？	3
4 实际实现并比较了哪些策略？	3
4.1 Baseline 和 Once-per-layer	3
4.2 A0：专用 Path，按名义需求分 CIR	3
4.3 A1：按层事件更新、用 CIR 偏向最早 deadline	3
4.4 B1：完全相同的 A1，再加在线 NPU 分配	3
5 核心结果：同一组变化请求，不挑赢家	4
5.1 同一完整请求集的延迟与尾部	4
6 为什么有的策略成功，有的失败？	5
6.1 自然错开没有一个“永远有效”的保证	5
6.2 Path 隔离和紧急度是不同的改进	5
6.3 消融与失败结果都保留	5
7 数学预测函数是否算得准？	5
8 控制成本、部署限制与推荐	6
9 文件与复现	6

1 先说结论：找到改善，但没有证实“总是大幅领先”

本轮确实运行了原项目仿真器，包含固定请求、逐请求改变 K/C/NQL 的输入，以及独立种子复核。**不是只用新写的预测函数自我验证。**

1. 固定的校准构造输入能使 Baseline 仅 **65.126%**，按需求分配 CIR 达 **100%**；但 Once-per-layer 已有 **98.990%**。这是比 Baseline 大幅改善，不是比 Once 大幅改善。
2. 直接从 data 取参数的可变输入中，简单 V/C → CIR 不是可靠赢家，因此继续实现了真正按层事件更新的 deadline 策略 A1。
3. 在长短请求混合 M1 中，Once 为 **92.902%**，固定 NPU 的 A1 为 **96.843%**，提升 **3.940 个百分点**。这相当于该一秒窗口中总 I/O 等待减少 **55.51%**，不是利用率相对增加这么多。
4. 最终组合 **B1 (A1 + 在线 fluid 分配)** 在四组原始参数变化输入中均优于 Once，利用率达到 **97.49%-99.98%**；M2 复核从 Once 的 **91.293%** 提高到 **97.812%**。四组完整请求集的 makespan、平均到达延迟和平均 stall 也均优于对应 A1；这是当前最值得继续验证的候选，不是一般最优性证明。A1 自身仍有反例，其他分配/速率启发式也保留失败案例。
5. **本轮没有在直接取自 data 的可变输入里找到 Baseline <80% 的例子。**此类输入测到的 Baseline 约 91%-96%。因此“真实变化请求下，Baseline 极低且新策略普遍接近满载”的完整目标尚未被证明。

data 是请求参数表，不是带用户身份、到达间隔和发生频率的线上 trace。以下称“原始参数”，不称“已证明真实流量分布”。请求采样权重和到达过程属于实验构造。

2 这次究竟固定了什么？

硬件：4 张独立 NPU、1 SSU、SSD 40 GiB/s、每卡独立接收链路 50 GiB/s。每条 I/O 固定为 1 个 KV block，即 **128 token 的单层 KV，共 176 KiB**；一次请求执行 8 层。跨请求预取开启，但晚于前请求末层计算开始才到达的请求不会补触发预取。

KV 数据路径为 SSD→HBM，无 DRAM 中转或预先 DRAM 缓存。256 条可用 QoS Path 仍共用一个物理 SSD，不会增加物理带宽。当前仿真器拒绝有限 PIR，因此 PIR 维持不限；没有假装实现一个尚不存在的限速器。

每组策略保持请求 ID、到达时间、总长、NQL、每层块数、计算时间、层数完全相同，并核验 input fingerprint。A 固定原 NPU 分配；B 只在真实到达时更改 NPU，不能迁移、改时刻或读取未来请求。原 sim.py、continuous_batch_sim.py、continuous_prefill_client.py、policy_logic.py 的哈希保持不变。

2.1 变化输入的一个具体参数表

M1/M2 请求池直接取原 data 的六行。每次请求从池里取一个完整画像，不单独修改计算时间后仍保留不匹配的 NQL。

总长度	NQL	每层 I/O 数	每层 MiB	每层计算 ms	名义 GiB/s
32K	128	255	43.82812	1.178026	36.3327
32K	256	254	43.65625	1.997480	21.3434
64K	512	508	87.31250	6.386487	13.3510
96K	512	764	131.31250	9.070842	14.1370
192K	512	1532	263.31250	17.123906	15.0165
192K	1024	1528	262.62500	34.100782	7.5209

“总长度”包括已有上下文与本次新 query。每层读取的历史 token 数为总长度减 NQL，再除以 128 得 I/O 数。例如第一行： $(32768 - 128)/128 = 255$ 条，数据为 255×176 KiB，计算时间直接取 data 的 1.178026 ms。

每张卡分别打乱配额循环。M1/M2 的六种画像配额为 8、4、4、4、8、16；L1/L2 使用 192K 的 NQL 512/1024/2048，配额为 17、14、1。同一张卡后续接到的请求会变化，并非永远重复同一画像。这些权重用于接近容量的压力实验，不是从参数表推断的用户概率。

2.2 三种“带宽”不要混在一起

单请求每层名义需求为 $d = V/(C/1000)$ 。混合请求每卡的长期理想需求必须按计算时间加权：

$$d_i^{\text{mix}} = \frac{\sum_{r \rightarrow i} 8V_r}{\sum_{r \rightarrow i} 8C_r/1000}.$$

它不是各个 V/C 的简单算术平均。全循环理论总需求：L 为 39.289 GiB/s，B 为 39.523 GiB/s；有限输入截断循环后，实际值见主表。“不超过 40”在主表指**原分配下完整有限请求集的理想时间加权需求**；不代表每个时刻四卡的需求和不超过 40，B 改分配后的组合也需重新理解。

另外，后续请求按 39.2 GiB/s 的字节预算安排到达，这是**到达过程**。开始时每卡还有 6 个已到达的积压请求，是额外启动突发，不能被排除后又声称整个输入到达率都不超过 40。第三种带宽是物理 SSD 的实际服务率，它始终受 40 GiB/s 约束。

输入	seed	每批请求数	请求总数	初始读取 GiB	含初始积压的速率
L1	20260906	1	63	49.307	63.332
L2	20260910	4	64	49.307	62.722
M1	20260906	1	80	34.217	56.170
M2	20260908	1	79	28.393	53.039

表末列为“全部请求读取字节数 ÷ 后续 pacing 预算时长”，包括启动积压，单位 GiB/s，所以大于 40。这是有积压系统的压力测试，不是从空系统开始始终平滑限额的生产流量复现。初始 KV 已在 SSD 上，不代表这些字节在仿真开始时同时经过 SSD 服务。

3 如何计算利用率，避免挑窗口？

可变输入全都完整运行到所有请求结束，然后统一取绝对 **1000-2000 ms**。对各层真实 compute 区间与窗口求交：

$$U_i = \frac{\sum_l |[S_{i,l}, E_{i,l}) \cap [1000, 2000)|}{1000}, \quad \bar{U} = \frac{U_0 + U_1 + U_2 + U_3}{4}.$$

每份结果核验四张卡在整个窗口都有已接纳请求，因此窗口内的非计算时间可归为 I/O barrier 等待，不是没任务。**有任务不等于一直计算**；stall 本身就是有任务却没数据。

再报告完整相同请求集的 makespan、到达至完成延迟、P99 和 I/O stall，防止 B 仅把更容易的请求移入测量窗口。全程利用率含启动及尾部空闲，不能与中间一秒的利用率直接混比。

4 实际实现并比较了哪些策略？

4.1 Baseline 和 Once-per-layer

Baseline 全部进同一个 FIFO Path0。Once 是仓库现有 layer_once，不是另写的弱化版本：每个请求层、每个 SSU 读取一次压力，按现有类别候选 Path 规划这一层的 I/O。它会做 Path 选择，但没有本轮 A1 的显式当前数据 deadline 与动态 CIR 优先方向。

4.2 A0：专用 Path，按名义需求分 CIR

四卡分别使用 Path 0、32、64、96。以 $d_i = V_i/C_i$ 规划 CIR，超出 40 时按比例缩放。更新发生在请求接纳、完成、跨请求预取边界；请求内每层画像固定，因此这一版**并不是每层重新计算的 deadline 控制**。

这延续项目已有 dedicated-Path Scheme B 架构，不是发明新 SSD。固定画像中有效；但可变请求切换时，下一请求 Layer0 的预算来自当前未层计算，不能一直用下一请求自己的 C。其他卡的进度以及已经读完的部分，也使单独 V/C 不足。

4.3 A1：按层事件更新、用 CIR 偏向最早 deadline

deadline_qos_controller.py 在实际层开始、数据到齐和请求边界更新。每张卡只提供当前待读层的：请求 ID、层号、未完成块数、数据 deadline、计算画像。数据 deadline 来自当前计算已经确定的结束时刻。

对仍有未完成块的层，选：

$$i^* = \arg \min_i D_i, \quad \text{CIR}_{i^*} = 40, \quad \text{CIR}_{j \neq i^*} = 0.$$

deadline 相同先选剩余工作少的层，再按 NPU ID 固定打破平局。任一层到齐或启动下一层时重新选择；**不必等一整条大请求结束**。队列空时其他零 CIR Path 可以借用盘，盘仍工作守恒。

这使另一个 Path 的新紧急读可以越过长流尚未服务的 FIFO 队尾，解除“所有卡共用 Path0 时无法绕过的前方工作”。已经正在服务的一条 I/O 仍不可抢占。下一层提前读完并不意味着 NPU 立刻换层，仍需等当前计算结束。

注意：CIR 是仲裁输入，不是直接把盘改成 EDF。原虚拟完成历史、未提交命令、HBM 尾和借用机制仍在，所以这里只称 deadline-CIR/EDF 倾向策略，不套用精确 EDF 最优性定理。

同一文件还实现 least_slack，用 $D_i - K_i s$ 排序。它只在层事件更新，不是持续更新的 LLF。

4.4 B1：完全相同的 A1，再加在线 NPU 分配

npu_request_assignment.py 的 fluid 模式在每次真实到达时，分别假设把新请求加入四张卡之一；用各卡已知未完成计算 c_i 和读取量 v_i 估计混合需求，并最小化：

$$\hat{T} = \max_i c_i \max \left(1, \frac{\sum_i v_i / (c_i / 1000)}{40} \right).$$

加入候选后再算各卡 c_i, v_i 。这兼顾计算积压和总读取容量，但仍是流体启发式，忽略具体层相位与未知后续到达；不能把它当精确完工时间。B1 与 A1 使用同一 deadline 存储策略，因此它们的差值才是增加分配权限的对照。

5 核心结果：同一组变化请求，不挑赢家

L1/M1 为首组，L2/M2 使用另一种子复核；L2 还把后续到达合成每批 4 请求。所有策略的原生提交随机种子也配对一致。输入代号 M1/M2 与策略代号 B1 不同。

输入	名义 GiB/s	Baseline	Once	A0	A1	B1
L1 长序列变化	38.006	95.685%	95.785%	95.198%	96.364%	99.985%
L2 长序列/成批到达	39.771	94.466%	94.459%	93.879%	94.347%	97.485%
M1 长短混合	38.568	93.564%	92.902%	95.488%	96.843%	98.377%
M2 长短混合复核	39.692	90.939%	91.293%	90.797%	93.394%	97.812%

上表 A0、A1 固定 NPU；B1 允许分配。**百分点**的变化不是“速度提升百分比”。例如 92.902% 到 96.843% 是 +3.940 个百分点。

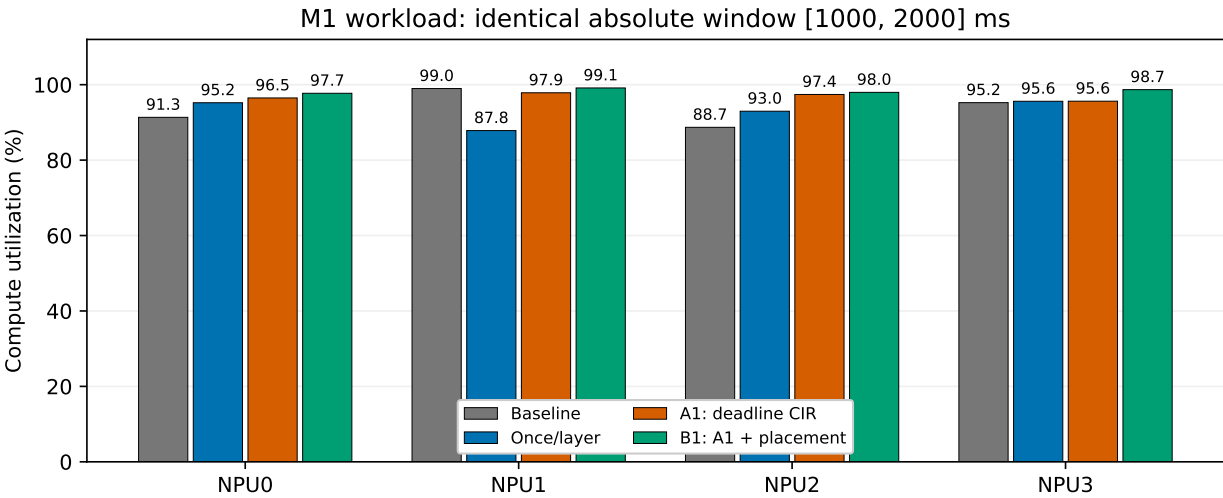


Figure 1: 同一 M1 输入、同一绝对一秒，按卡核对计算占比；柱顶是数值，不表示 SSD 实际服务份额。

5.1 同一完整请求集的延迟与尾部

“平均延迟”从实际到达算到完成，包含 admission 排队；“平均等待”是每请求从接纳到完成扣去自身计算时间，包含首层读料。不是之前教程仅在窗口内接纳并标记的样本数。

输入	策略	全程利用率	全完工 ms	平均延迟 ms	P99 ms	平均等待 ms
L1	Once/layer	90.40%	3776.36	1259.18	1947.15	9.44
L1	A1 : deadline	89.63%	3808.78	1242.03	1981.81	7.99
L1	B1 : A1+ 分配	94.23%	3622.79	1190.17	1777.03	4.50
L2	Once/layer	87.18%	3799.33	1324.72	1878.68	15.13
L2	A1 : deadline	88.30%	3751.03	1321.32	1859.33	15.29
L2	B1 : A1+ 分配	91.03%	3638.65	1290.52	1763.50	12.94
M1	Once/layer	81.38%	3600.74	1006.71	1712.11	12.43
M1	A1 : deadline	84.99%	3447.61	957.47	1558.98	7.96
M1	B1 : A1+ 分配	91.70%	3195.57	904.75	1235.06	6.10
M2	Once/layer	83.38%	3291.79	810.02	1348.48	11.73
M2	A1 : deadline	85.97%	3192.60	784.80	1256.82	8.87
M2	B1 : A1+ 分配	92.70%	2961.06	733.38	1027.29	5.13

这张表用于识别提高平均利用率却牺牲某些请求/P99 的情况。把延迟目标换成 admission 后 TTFT 会隐藏入口积压；不能只报告后一种 SLO。这里选中间一秒作为主指标，不宣称调度对所有 SLO 都最优。

6 为什么有的策略成功，有的失败？

6.1 自然错开没有一个“永远有效”的保证

真正的条件是读数时刻是否早于计算需求时刻，而不是突发图上是否看起来错开。FIFO 中若某批完整入队后还有 H ms 工作，短预算卡每层计算 C 、自己读取需 w ms、接收尾为 ℓ ，且仍有足够后续层，则：

$$H + w + \ell > 2C$$

是迫使它后续至少一次 stall 的**相位无关充分条件**。无 stall 时下一次预取必须在 C 内发起，并在再一个 C 内到齐；但前面不可越过的工作已超过这个上限。详见配套数学 PDF 的逐步证明。条件不成立不代表一定没有 stall，证书也不直接给出整窗平均利用率。

原始 data 的最快可用小请求仍有实际计算成本。两个 (32K, 256) 仅名义需求就超过 40 GiB/s，不能任意安排多张极短计算卡，再声称总需求仍小于盘容量。这也是纯 raw 固定画像难复现旧构造极低平均值的一个原因。

6.2 Path 隔离和紧急度是不同的改进

Once 已经能分散队列，所以旧案例 Once 接近满载。对它的额外改进不能归因于“新策略有 256 条 Path，而 Once 没有”。本轮 A 只用四条，关键是**独立所有权和随实际 deadline 改变的服务优先方向**，不是 Path 数越多越好。

在变化输入中，同类别不等于相同计算预算；统计平均 V/C 也不表示现在还剩多少时间。A1 按实际计算结束时刻和未完成层判断紧急度，避免把已到齐的数据继续当作急需整层带宽。这解释了为什么它能在长短混合中超过 A0/Once。但 L2 中 A1 仍略输 Once：瞬时容量不足、优先完成哪张卡、后续预取相位与尾延迟之间仍有冲突。

这些是受控消融支持的机制解释，不是对每次误差已逐 I/O 做完唯一因果归因；原调度器保留的虚拟完成历史也是替代解释变量。

6.3 消融与失败结果都保留

策略	L1	L2	M1	M2
等分 CIR	94.878%	未测	95.676%	未测
A0 : V/C	95.198%	93.879%	95.488%	90.797%
切换预算	95.197%	未测	95.541%	未测
预算 + 流体利用率	97.292%	91.404%	95.396%	91.388%
最小松弛度	95.875%	94.018%	96.434%	94.275%
A1 : deadline	96.364%	94.347%	96.843%	93.394%

“预算 + 流体利用率”先保留比例分配的一半，再优先满足低需求卡；它在静态流体模型可提高计算占比，但 L2 反而显著下降，个别卡会被牺牲。没有因为首组漂亮就设为默认。

“切换预算”把 successor 的读量除以前请求末层的 C ；这修正了分母语义，但单独改这个公式并不保证收益。请求变化时需要进度/到齐事件，而不是只修改一个长期带宽数字。

7 数学预测函数是否算得准？

npu_stall_predictor.py 是独立、标准库实现。三层功能要分清：

- screen_current_layer：在目标层完全尚未提交时，用 Path 总 outstanding 数和在途剩余量的已知/未知界限筛查；默认逐条提交，只能有条件保证。
- forecast_path0/predict_request_impact：从各卡当前计算进度、下一待计算层的 queued/unissued/ready 状态，以及已到达待接纳请求递推；未知 FIFO 顺序按声明的场景重建。
- fifo_burst_certificate：给出上述自然错开不能完全消除等待的充分条件。

validate_stall_predictor.py 与原生仿真逐层对照：单卡误差 0；四卡空队列冷启动最大误差约 4.196–8.392 微秒；动态非空快照的到齐误差达到 0.461121 ms、计算开始/未来 stall 误差 0.079269 ms。已有请求延迟增量在该样本中误差 0.003738 ms，但不代表所有输入都这么准。

仅凭 Path 总排队数无法确定队内所有者及其他卡的下一次释放。若要多层预测，必须补各卡当前计算剩余时间、当前层提交/完成计数和已到达队列。它没有精确预测 256 Path 的底层虚拟完成标签；**A1 使用可观测 deadline 算术直接决策，不谎称用了完整预测器获得精确最优调度。**

8 控制成本、部署限制与推荐

下面是 M1 输入完整运行中的计数，不是一秒内的次数。Once 没有 CIR 写不代表没有路径规划 CPU 成本。

策略	压力读取	控制计算	CIR 提交	Path 项写入
Once/layer	640	0	0	0
A0 : V/C	156	156	61	205
A1 : deadline	1360	1360	922	1652
B1 : A1+ 分配	1360	1360	949	1778

A1 在 layer-ready 和 layer-start 都可能触发，约为每层两次控制机会，另加请求边界；比原 Once 的控制内容更重。CIR 可能反复在 0/40 间切换。上述仿真统计了次数，但**没有把硬件读取、远程通知、CIR 写入和 Python 计算耗时加入仿真时间**。当前改动是实验适配器，不是生产 RPC/驱动。

真实部署至少需要：各 NPU 一致时间口径的 compute-end/deadline；已释放层未完成块数；层开始/到齐通知；Path→NPU 所有权；CIR 更新 API。A1 不需要 SSU FIFO 顺序或活动 I/O 剩余字节。未完成数包含未提交块及可能的一条 HBM 尾，这是客户端可维护的信息，不冒充精确 SSD queue 深度。

建议以 **A1 deadline-CIR 为下一轮固定 NPU 候选**，以 **B1 为允许分配时的候选**，而非直接替换 Once。下一步优先验证控制延迟/限频、计算抖动、更多用户相关性与真实到达 trace；再同时约束平均利用率和请求 P99。未建模的 GC、介质内部并行与驱动拆分等也可能改变收益。仿真数据不能证明实际 SSD 已支持此效果。

9 文件与复现

项目根目录运行。主目录只保留最终 PDF，中间数据、图、说明分别在 data/figures/docs。

```
python -m unittest discover -q
python validate_stall_predictor.py
python run_stall_policy_experiments.py --case calibrated_two_ls
python run_stall_policy_experiments.py --case variable_raw \
  --mix broad --seed 20260906 \
  --strategies baseline layer_once dedicated_demand
python run_layer_budget_experiments.py --mix broad \
  --seed 20260906 --modes edf
python run_layer_budget_experiments.py --mix broad \
  --seed 20260906 --modes edf --assignment fluid
python build_stall_experiment_report.py
```

核心入口：npu_stall_predictor.py(公式),deadline_qos_controller.py(A1),npu_request_assignment.py (B 的分配), run_layer_budget_experiments.py (相同输入接线)。其余候选保留用于消融，不要求生产部署全部文件。

data/experiment_index.csv 汇总所有可变输入实测，data/report_audit.json 记录主表来源与哈希。旧 B 探索曾误用默认提交 seed42；已保留并标为 seed 未匹配，**不进入本报告严格配对主表**，另用相同种子重新运行。轻微过载的 L/20260907 也保留，但不列为 ≤40 主案例。

本轮保留旧教程与原 trace，不覆盖历史结论。所有数字都来自保存的仿真输出；公式为限定模型的直接推导，现实频率、硬件收益和全局最优性均未作事实断言。